Agentic架构5: 多智能体系统（Multi-Agent Systems）

> **多智能体系统（Multi-Agent Systems）**——超越了单一智能体（无论其多么复杂）的概念，转而构建一个**由多个专业智能体组成的团队**，通过**协作**来解决问题。每个智能体都拥有**明确的角色、人设和技能组合**，与人类专家团队的工作方式相呼应。实现了**深度的“分工协作”，将复杂问题分解为若干子任务，并分配给最擅长处理该任务的智能体**。

定义

多智能体系统 是一种架构，其中**多个具有不同专长的智能体**通过协作（有时也存在竞争）来实现一个共同目标。系统通过一个**中央控制器**或定义好的**工作流协议**来管理智能体之间的通信与任务路由。

高级工作流程

1. **任务分解：** 主控制器或用户提出一个**复杂任务**。
2. **角色定义：** 系统根据**预定义的智能体角色**（例如“研究员”、“程序员”、“评审员”、“撰写者”），将子任务分配给对应的**专业智能体**。
3. **协作：** 各智能体执行其任务，通常是**并行或顺序执行**。它们将输出结果相互传递，或发送至中央“黑板”。
4. **整合：** 最终由 **“管理者”或“合成者”智能体**收集所有专业智能体的输出，并将其整合成最终的、统一的响应。

适用场景 / 应用

- **复杂报告生成：** 创建需要**多领域专业知识**支撑的详细报告（例如财务分析、科学研究）。
- **软件开发流程：** 模拟一个包含程序员、代码审查员、测试员和项目经理的**开发团队**。
- **创意头脑风暴：** 一组拥有不同 **“性格”**（例如乐观型、谨慎型、天马行空型）的智能体可以碰撞出更多元化的创意。

优势与劣势

- **优势：**
  - **专业化与深度：** 每个智能体都可以通过**特定的人设和工具**进行**微调**，从而在其**专长领域**产出更高质量的结果。
  - **模块化与可扩展性：** 无需重新设计整个系统，即可轻松添加、移除或升级单个智能体。
  - **并行处理：** 多个智能体可以**同时处理各自的子任务**，从而可能减少整体任务耗时。
- **劣势：**
  - **协调开销：** 管理智能体间的**通信和工作流**会增加系统设计的复杂性。
  - **成本与延迟增加：** 运行多个智能体意味着**更多的大语言模型调用次数**，这比单智能体方案的成本更高、速度也可能更慢。

阶段 0：基础与环境搭建

首先安装所需的库，并为 Nebius、LangSmith 和 Tavily 配置 API 密钥。

步骤 0.1：安装核心库

安装本系列项目所需的标准库套件

```
pip install -q -U langchain-nebius langchain langgraph rich python-dotenv
langchain-tavily
```

步骤 0.2：导入库并配置密钥

导入必要的模块，并从 `.env` 文件中加载 API 密钥。

需要操作： 在此目录下创建一个 `.env` 文件，并填入密钥：

```
NEBIUS_API_KEY="你的_nebius_api_密钥"
LANGCHAIN_API_KEY="你的_langsmith_api_密钥"
TAVILY_API_KEY="你的_tavily_api_密钥"
```

```
"""
Agentic架构 - 多智能体系统基础模块
该模块实现了基于LangGraph的多智能体协同框架，集成Nebius LLM和Tavily搜索工具
"""

import os
from typing import List, Annotated, TypedDict, Optional
from dotenv import load_dotenv

# ==================== LangChain 组件导入 ====================
# LangChain提供了LLM抽象、工具集成和提示模板等核心功能
from langchain_nebius import ChatNebius  # Nebius云平台的大语言模型接口
from langchain_tavily import TavilySearch  # Tavily搜索引擎工具，用于获取实时网
络信息
from langchain_core.messages import BaseMessage, SystemMessage,
HumanMessage  # 消息类型定义
from pydantic import BaseModel, Field  # Pydantic用于数据验证和配置管理
from langchain_core.prompts import ChatPromptTemplate  # 结构化提示模板

# ==================== LangGraph 组件导入 ====================
# LangGraph提供基于图的状态机框架，用于构建复杂的多智能体工作流
from langgraph.graph import StateGraph, END  # 状态图核心组件
from langgraph.graph.message import AnyMessage, add_messages  # 消息状态管理
from langgraph.prebuilt import ToolNode, tools_condition  # 预构建的工具节点和
条件路由

# ==================== 可视化与调试工具 ====================
# Rich库提供增强的终端输出格式，便于调试和演示
from rich.console import Console  # 富文本控制台输出
from rich.markdown import Markdown  # Markdown格式渲染

# ==================== API密钥与环境配置 ====================
"""
```

```
环境变量配置说明:
- NEBIUS_API_KEY: Nebius LLM服务的认证密钥
- LANGCHAIN_API_KEY: LangChain追踪服务的认证密钥
- TAVILY_API_KEY: Tavily搜索API的访问密钥
- LANGCHAIN_PROJECT: LangChain追踪项目标识符
"""

load_dotenv()  # 从.env文件加载环境变量

# 配置LangChain追踪系统，用于监控和调试智能体行为
os.environ["LANGCHAIN_TRACING_V2"] = "true"  # 启用V2版本的追踪功能
os.environ["LANGCHAIN_PROJECT"] = "Agentic Architecture - Multi-Agent
(Nebius)"  # 设置追踪项目名称

# 验证关键环境变量是否存在，缺失时给出提示
for key in ["NEBIUS_API_KEY", "LANGCHAIN_API_KEY", "TAVILY_API_KEY"]:
    if not os.environ.get(key):
        print(f"{key} not found. Please create a .env file and set it.")

print("Environment variables loaded and tracing is set up.")
```

阶段 1：基线对比——单一“通才型”智能体

为了凸显专家团队的价值，首先需要观察**单一智能体**在复杂任务上的表现。构建一个 **ReAct 智能体**，并赋予它一个**宽泛的提示词**，要求它同时执行多种类型的分析。

步骤 1.1：构建通才型智能体

构建一个**标准的 ReAct 智能体**。为其配备一个**网络搜索工具**，并设置一个**非常通用的系统提示词**，要求其扮演一个全面的财务分析师角色。

```
# ==================== 控制台输出初始化 ====================
# 初始化Rich控制台实例，用于增强终端输出格式和可视化效果
console = Console()

# ==================== 智能体共享状态定义 ====================
# 定义多智能体系统中共用的状态数据结构
# 该状态图遵循LangGraph的消息传递模式，支持状态在智能体节点间流转
class AgentState(TypedDict):
    """
    智能体状态数据结构
    属性:messages: 消息列表，使用add_messages reducer实现增量更新
                  Annotated类型指示LangGraph如何处理状态合并
    """
    messages: Annotated[list[AnyMessage], add_messages]  # 消息历史记录，支持
增量追加

# ==================== 工具与大语言模型初始化 ====================
# 配置智能体可调用的外部工具和底层LLM服务
search_tool = TavilySearch(
```

```
    max_results=3,            # 限制搜索结果数量，平衡信息质量与token消耗
    name="web_search"         # 工具标识符，用于LLM的函数调用
)

# 初始化Nebius LLM实例，配置模型参数
llm = ChatNebius(
    model="meta-llama/Meta-Llama-3.1-8B-Instruct",     # 使用8B参数的指令微调
模型
    temperature=0                                      # 温度设为0确保输出确定
性，适用于工具调用场景
)

# 将工具绑定到LLM，使其具备函数调用能力
llm_with_tools = llm.bind_tools([search_tool])

# ==================== 单智能体节点定义 ====================
# 定义单智能体架构中的核心处理节点
def monolithic_agent_node(state: AgentState):
    """
    单智能体节点处理函数 - 实现ReAct架构中的"思考"步骤
    工作流程:
    1. 接收当前状态中的消息历史
    2. 调用绑定了工具的LLM进行推理
    3. 返回LLM响应（可能包含工具调用请求）

    参数:state: 当前智能体状态，包含消息历史
    返回:dict: 包含LLM响应的状态更新字典
    """
    console.print("--- MONOLITHIC AGENT: Thinking... ---")  # 可视化处理状态
    response = llm_with_tools.invoke(state["messages"])     # LLM推理与工具调
用决策
    return {"messages": [response]}                         # 将响应追加到消
息历史

# 创建工具执行节点，用于处理LLM产生的工具调用请求
tool_node = ToolNode([search_tool])

# ==================== ReAct图结构构建 ====================
# 构建基于ReAct（Reasoning + Acting）范式的计算图，定义了智能体的循环推理-执行工作流
mono_graph_builder = StateGraph(AgentState)  # 初始化状态图构建器

# 添加节点：思考节点（Agent）和执行节点（Tools）
mono_graph_builder.add_node("agent", monolithic_agent_node)  # 推理节点
mono_graph_builder.add_node("tools", tool_node)              # 工具执行节点
mono_graph_builder.set_entry_point("agent")                  # 设置入口点为
agent节点

# ==================== 条件路由函数 ====================
# 定义智能体节点后的路由逻辑，决定下一步执行路径
def tools_condition_with_end(state):
    """
    增强版工具条件判断函数 - 实现ReAct循环的终止控制
    根据LLM响应决定:
    - 如果LLM请求调用工具：路由到tools节点
```

```
    - 如果LLM直接回复：终止执行流程
    封装了langgraph.prebuilt.tools_condition的逻辑，并添加了明确的终止条件处理。

    参数:state: 当前状态，包含最新的LLM响应消息
    返回:dict: 节点路由映射，包含默认的END终止条件
    """
    result = tools_condition(state)  # 调用内置条件判断函数

    # 处理不同版本的返回值类型兼容性
    if isinstance(result, str):
        # 旧版本返回字符串类型："tools" 或 "agent"
        return {result: "tools", "__default__": END}
    elif isinstance(result, dict):
        # 新版本返回字典映射，添加默认终止条件
        result["__default__"] = END
        return result
    else:
        raise TypeError(f"Unexpected type from tools_condition:
{type(result)}")

# 添加条件边：根据agent输出决定下一步流向
mono_graph_builder.add_conditional_edges(
    "agent",                        # 源节点
    tools_condition_with_end        # 条件判断函数
)

# 添加固定边：工具执行完成后返回agent节点继续推理
mono_graph_builder.add_edge("tools", "agent")

# ==================== 图编译与应用初始化 ====================
# 将构建的图结构编译为可执行的应用实例
monolithic_agent_app = mono_graph_builder.compile()
print("Monolithic 'generalist' agent compiled successfully.")
```

步骤 1.2：测试通才型智能体

给这个通才型智能体一个复杂任务：为一家公司创建一份完整的市场分析报告，涵盖三个不同的领域。

```
# ==================== 测试用例配置 ====================
# 定义待分析的标的公司，用于构建多维度金融分析任务
company = "NVIDIA (NVDA)"

# 构建综合查询提示词 - 模拟真实的金融分析工作流
monolithic_query = f"""Create a brief but comprehensive market analysis
report for {company}.
The report should include three sections:
1. A summary of recent news and market sentiment.
2. A basic technical analysis of the stock's price trend.
3. A look at the company's recent financial performance."""

# ==================== 测试执行与可视化 ====================
```

```
# 输出测试开始标识，使用Rich的彩色格式化提升可读性
console.print(
    f"[bold yellow]Testing MONOLITHIC agent on a multi-faceted task:[/bold
yellow]\n'{monolithic_query}'\n"
)

# ==================== 单智能体系统调用 ====================
# 执行单智能体图应用，处理多维度金融分析任务
# 该调用将触发ReAct循环：推理 -> 工具调用 -> 再推理 -> 最终输出
final_mono_output = monolithic_agent_app.invoke({
    "messages": [
        # 系统提示：定义智能体的角色和专业定位
        # 使用SystemMessage设置智能体的行为边界和任务目标
        SystemMessage(
            content="You are a single, expert financial analyst. You must
create a comprehensive report covering all aspects of the user's request."
        ),
        # 用户提示：包含具体的分析任务和输出格式要求
        HumanMessage(content=monolithic_query)
    ]
})

# ==================== 结果输出与渲染 ====================
# 输出最终报告的分隔标识
console.print("\n--- [bold red]Final Report from Monolithic Agent[/bold
red] ---")

# 使用Markdown渲染智能体生成的报告内容
# 通过索引-1获取最后一条消息（即智能体的最终响应）
console.print(Markdown(final_mono_output['messages'][-1].content))
```

> **输出结果讨论：** 通才型智能体生成了一份报告。它可能进行了**多次网络搜索**，并尽力整合了信息。然而，输出结果可能存在以下不足：
>
> - **缺乏结构：** 各部分内容可能混杂在一起，没有清晰的标题或专业的格式。
> - **分析浅显：** 试图同时成为三个领域的专家，导致智能体可能只提供了高层次的概括，在任何一个领域都缺乏深度。
> - **语言风格泛化：** 表述可能较为通用，缺乏各领域真正专家所使用的专业术语和针对性视角。
>
> 这个结果作为**基线**。它能够完成任务，但称不上出色。

阶段 2：进阶方案——多智能体专家团队

组建一个**专家团队**：包括新闻分析师、技术分析师和财务分析师。每个分析师都将作为一个**独立的智能体节点**，拥有**特定的角色定位**。最后，由报告撰写者作为管理者，负责整合他们的分析成果。

步骤 2.1：定义专业智能体节点

创建三个不同的智能体节点。其核心区别在于为每个节点设置了**高度具体的系统提示词**。这些提示词**定义了智能体的角色定位、专业领域以及输出结果应遵循的精确格式**。通过这种方式实现**专业化的分工**。

```python
# ==================== 多智能体系统状态定义 ====================
# 定义多智能体协同系统中的状态数据结构
# 该状态采用"分而治之"的设计模式，每个专家独立生成报告片段
# 最终由报告编写者整合为完整输出
class MultiAgentState(TypedDict):
    """
    多智能体系统状态数据结构
    该状态设计遵循数据流架构原则：
    - user_request: 输入层的原始用户请求（驱动整个系统）
    - 各专家报告: 并行计算层产生的中间结果（Optional类型允许缺失）
    - final_report: 输出层的综合结果
    """
    user_request: str                       # 原始用户查询，作为各专家的共同输入
    news_report: Optional[str]              # 新闻分析专家输出
    technical_report: Optional[str]         # 技术分析专家输出
    financial_report: Optional[str]         # 财务分析专家输出
    final_report: Optional[str]             # 综合报告编写者输出

# ==================== 专家节点工厂函数 ====================
def create_specialist_node(persona: str, output_key: str):
    """
    专家智能体节点工厂函数
    实现了"函数即服务"的设计模式，动态创建专业化的分析节点。通过闭包捕获persona和output_key，为每个专家生成独立的处理逻辑。

    参数:persona: 专家的角色描述，定义其专业领域和行为边界
        output_key: 输出字段名，用于在状态中存储该专家的分析结果
    返回:callable: 符合LangGraph节点接口的处理函数
    """

    # 构建系统提示词 - 定义专家的角色定位和输出规范
    # 明确要求输出Markdown格式，确保各专家输出的一致性
    system_prompt = persona + "\n\nYou have access to a web search tool. Your output MUST be a concise report section, formatted in markdown, focusing only on your area of expertise."

    # 使用ChatPromptTemplate构建结构化提示模板
    # 相比原始的消息列表，模板模式提供：更清晰的输入参数定义+自动化的消息格式化+更好的可组合性
    prompt_template = ChatPromptTemplate.from_messages([
        ("system", system_prompt),      # 系统消息：设定角色边界
        ("human", "{user_request}")     # 用户消息：定义输入占位符
    ])

    # 使用管道操作符(|)组合提示模板和绑定工具的LLM，实现了可组合、可流式处理的处理链
    agent = prompt_template | llm_with_tools

    def specialist_node(state: MultiAgentState):
        """
        专家节点执行函数
        参数:state: 当前多智能体状态
        返回值:dict: 包含专家分析结果的状态更新
```

```
        """
        # 输出执行状态，便于调试和可视化
        console.print(f"--- CALLING
{output_key.replace('_report','').upper()} ANALYST ---")

        # 调用处理链，传入用户请求
        result = agent.invoke({"user_request": state["user_request"]})

        # 处理LLM响应：优先使用content字段，若为空则记录工具调用信息
        # 这种处理确保了即使LLM只返回工具调用请求，也能获得有意义的输出
        content = result.content if result.content else f"No direct
content, tool calls: {result.tool_calls}"

        # 返回状态更新，LangGraph会自动合并到全局状态
        return {output_key: content}

    return specialist_node


# ==================== 专家节点实例化 ====================
# 使用工厂函数创建三个专业分析节点，每个节点拥有不同的角色定位和专业知识领域

# 1. 新闻分析专家：负责信息搜集和市场情绪分析
news_analyst_node = create_specialist_node(
    "You are an expert News Analyst. Your specialty is scouring the web for
the latest news, articles, and social media sentiment about a company.",
    "news_report"
)
# 2. 技术分析专家：负责价格趋势和技术指标分析
technical_analyst_node = create_specialist_node(
    "You are an expert Technical Analyst. You specialize in analyzing stock
price charts, trends, and technical indicators.",
    "technical_report"
)
# 3. 财务分析专家：负责财务报表和基本面分析
financial_analyst_node = create_specialist_node(
    "You are an expert Financial Analyst. You specialize in interpreting
financial statements and performance metrics.",
    "financial_report"
)

# ==================== 报告整合节点 ====================
def report_writer_node(state: MultiAgentState):
    """
    报告编写者节点 - 多智能体系统的聚合层
    这是"分治-整合"(Divide and Conquer)架构的关键组件，将多个专家的输出转化为用户可
读的最终产品。

    参数:state: 包含所有专家报告的多智能体状态
    返回:dict: 包含最终综合报告的状态更新
    """
    console.print("--- CALLING REPORT WRITER ---")

    # 构建整合提示词 - 将所有专家报告组合为统一的输入
```

```
    # 使用f-string将各字段内容嵌入到提示模板中
    prompt = f"""You are an expert financial editor. Your task is to
combine the following specialist reports into a single, professional, and
cohesive market analysis report. Add a brief introductory and concluding
paragraph.

    News & Sentiment Report:
    {state['news_report']}

    Technical Analysis Report:
    {state['technical_report']}

    Financial Performance Report:
    {state['financial_report']}
    """

    # 调用基础LLM（不带工具）执行整合任务
    # 这里使用llm而非llm_with_tools，因为整合任务不需要额外搜索
    final_report = llm.invoke(prompt).content

    return {"final_report": final_report}

print("Specialist agent nodes and Report Writer node defined.")
```

步骤 2.2：构建多智能体工作流图

将**各个专业智能体与管理者连接成一个工作流图**。对于本次任务而言，各专业智能体可以独立工作，因此我们可以按照简单的顺序**依次执行**它们（在实际应用场景中，这些步骤可以**并行运行**）。**工作流的最终步骤始终是报告撰写者**。

```
# ==================== 多智能体系统图构建 ====================
# 初始化多智能体状态图，使用MultiAgentState作为状态容器，该状态图将实现专家团队的串行
协作模式
multi_agent_graph_builder = StateGraph(MultiAgentState)

# ==================== 添加专家节点 ====================
# 将三个专业分析节点和报告整合节点添加到计算图中，每个节点代表一个独立的智能体，拥有特定的
专业领域知识

# 添加新闻分析专家节点 - 负责信息搜集和市场情绪分析
multi_agent_graph_builder.add_node("news_analyst", news_analyst_node)
# 添加技术分析专家节点 - 负责价格趋势和技术指标分析
multi_agent_graph_builder.add_node("technical_analyst",
technical_analyst_node)
# 添加财务分析专家节点 - 负责财务报表和基本面分析
multi_agent_graph_builder.add_node("financial_analyst",
financial_analyst_node)
# 添加报告编写节点 - 负责整合所有专家输出为综合报告
# 该节点充当"总编辑"角色，确保输出的连贯性和完整性
multi_agent_graph_builder.add_node("report_writer", report_writer_node)
```

```
# ==================== 定义工作流执行序列 ====================
# 该多智能体系统采用串行流水线架构（Pipeline Architecture）,执行流程遵循严格的顺序依赖关系，每个节点的输出为后续节点提供输入

# 设置入口点：从新闻分析开始
# 设计决策：新闻分析作为第一步，因为其他分析可能需要实时市场信息作为参考
multi_agent_graph_builder.set_entry_point("news_analyst")

# 定义节点间的执行顺序 - 串行依赖链
# 新闻分析完成后，将状态传递给技术分析专家
multi_agent_graph_builder.add_edge("news_analyst", "technical_analyst")
# 技术分析完成后，将状态传递给财务分析专家
multi_agent_graph_builder.add_edge("technical_analyst",
"financial_analyst")
# 所有专业分析完成后，将状态传递给报告编写者进行整合
multi_agent_graph_builder.add_edge("financial_analyst", "report_writer")

# 报告编写完成后，终止执行流程
# END是LangGraph预定义的终止节点标识
multi_agent_graph_builder.add_edge("report_writer", END)

# ==================== 图编译与应用实例化 ====================
# 将定义好的图结构编译为可执行的应用实例
# 编译过程会进行图验证，确保：所有节点都已定义+执行路径完整无循环+状态传递正确
multi_agent_app = multi_agent_graph_builder.compile()
print("Multi-agent specialist team compiled successfully.")
```

阶段 3：直接对比

让专家团队执行与通才型智能体完全相同的任务，并对最终生成的报告进行比较。

```
# ==================== 多智能体系统测试用例配置 ====================
# 构建与单智能体系统相同的分析任务，用于对比评估，保持输入一致性是进行公平性能比较的关键
company = "NVIDIA (NVDA)"  # 沿用之前定义的标的公司

# 构建用户查询 - 与单智能体系统使用相同的任务描述
# 这种设计允许进行A/B测试，比较两种架构的输出质量
multi_agent_query = f"Create a brief but comprehensive market analysis
report for {company}."

# ==================== 初始化输入状态 ====================
# 构建多智能体系统的初始状态
# 注意：与单智能体系统不同，多智能体系统期望的是结构化状态输入
initial_multi_agent_input = {"user_request": multi_agent_query}

# ==================== 测试执行与可视化 ====================
# 输出测试开始标识，使用绿色区分单智能体测试的黄色标识
# Rich的彩色格式化提供了直观的测试阶段视觉反馈
console.print(
    f"[bold green]Testing MULTI-AGENT TEAM on the same task:[/bold
```

```
green]\n'{multi_agent_query}'\n"
)

# ==================== 多智能体系统调用 ====================
# 执行多智能体图应用，触发专家团队协作流程
# 该调用将依次执行以下节点链：
# news_analyst → technical_analyst → financial_analyst → report_writer
final_multi_agent_output =
multi_agent_app.invoke(initial_multi_agent_input)

# ==================== 结果输出与渲染 ====================
# 输出最终报告的分隔标识，使用绿色与单智能体的红色区分
console.print("\n--- [bold green]Final Report from Multi-Agent Team[/bold
green] ---")

# 使用Markdown渲染多智能体生成的综合报告，与单智能体系统的输出格式保持一致，便于并排比较
console.print(Markdown(final_multi_agent_output['final_report']))
```

> **输出结果讨论：** 最终报告的差异十分显著。多智能体团队产出的报告具备以下特点：
>
> - **结构清晰：** 报告为每个分析领域都设有**明确的、独立的部分**，因为每个部分都由遵循**特定格式要求**的专业智能体生成。
> - **分析深入：** 每个部分都包含了**更详细、更具领域专业性**的术语和洞见。技术分析师会讨论移动平均线，新闻分析师会探讨市场情绪，而财务分析师则聚焦于收入和盈利情况。
> - **专业性更强：** 由报告撰写者整合而成的最终报告，读起来像一份**专业文档**，包含清晰的引言、正文和结论。
>
> 通过这种定性对比可以看出，通过**将任务分工给一个专家团队**，可以获得了比单一通才型智能体更优越的结果，而这种高质量的输出是后者难以企及的。

阶段 4：定量评估

为了将对比结果形式化，采用“**大语言模型即评审**”的方式对两份报告进行评分。评估标准将聚焦于预期多智能体方案表现更优的维度，例如**结构清晰度和分析深度**。

```
# ==================== 评估模型定义 ====================
# 使用Pydantic定义结构化输出模型，用于标准化报告质量评估，该模型确保评估结果具有一致的数
据结构，便于后续分析和比较
class ReportEvaluation(BaseModel):
    """
    金融报告质量评估模型
    该评估框架从三个维度对报告进行量化评分：
    1. 清晰度与结构：评估报告的组织方式和可读性
    2. 分析深度：评估专业分析的透彻程度和质量
    3. 完整性：评估报告对用户需求的覆盖程度
    """
    # 评分维度1：评估报告的逻辑结构、段落组织和信息呈现的清晰度
    clarity_and_structure_score: int = Field(
        description="Score 1-10 on the report's organization, structure,
and clarity."
```

```
    )
    # 评分维度2：评估分析的透彻程度、论证的充分性和洞察的深度
    analytical_depth_score: int = Field(
        description="Score 1-10 on the depth and quality of the analysis in
each section."
    )
    # 评分维度3：评估报告是否完整覆盖用户请求的所有方面
    completeness_score: int = Field(
        description="Score 1-10 on how well the report addressed all parts
of the user's request."
    )
    # 定性反馈：提供评分的详细说明，增强评估的可解释性
    justification: str = Field(
        description="A brief justification for the scores."
    )

# ==================== 评估LLM配置 ====================
# 将基础LLM配置为结构化输出模式
# with_structured_output方法将LLM的输出强制转换为指定的Pydantic模型
judge_llm = llm.with_structured_output(ReportEvaluation)

# ==================== 评估函数定义 ====================
def evaluate_report(query: str, report: str) -> ReportEvaluation:
    """
    报告质量评估函数 - 作为评估智能体
    评估方法论：
    - 使用原始查询作为评估基准，确保评分与任务目标对齐
    - 三个评分维度覆盖了内容质量的不同方面
    - 结合量化评分和定性说明，提供全面的评估反馈

    参数:query: 原始用户请求，作为评估的参考标准
        report: 待评估的报告内容
    返回:ReportEvaluation: 包含三个维度的评分和详细说明的结构化评估结果
    """

    # 构建评估提示词 - 明确定义评估标准和格式要求，使用Markdown格式清晰分隔输入的不同
部分
    prompt = f"""You are an expert judge of financial analysis reports.
Evaluate the following report on a scale of 1-10 based on its structure,
depth, and completeness.

    **Original User Request:**
    {query}

    **Report to Evaluate:**\n
    {report}
    """

    # 调用结构化LLM进行评估
    # invoke方法返回ReportEvaluation实例，自动完成JSON解析和Pydantic验证
    return judge_llm.invoke(prompt)

# ==================== 单智能体系统评估 ====================
# 对单智能体生成的报告进行质量评估，该评估提供了与多智能体系统对比的基准
```

```
console.print("--- Evaluating Monolithic Agent's Report ---")

# 调用评估函数，传入原始查询和单智能体输出的最终报告
mono_agent_evaluation = evaluate_report(
    monolithic_query,                                    # 原始用户查询
    final_mono_output['messages'][-1].content            # 单智能体报告内容
)

# 输出评估结果 - 使用model_dump()将Pydantic模型转换为字典格式
console.print(mono_agent_evaluation.model_dump())

# ==================== 多智能体系统评估 ====================
# 对多智能体团队生成的报告进行质量评估，与单智能体评估使用相同的标准和流程，确保公平比较
console.print("\n--- Evaluating Multi-Agent Team's Report ---")

# 调用评估函数，传入原始查询和多智能体团队输出的最终报告
multi_agent_evaluation = evaluate_report(
    multi_agent_query,                                   # 原始用户查询
    final_multi_agent_output['final_report']             # 多智能体综合报告
)

# 输出评估结果，保持与单智能体评估相同的输出格式，这便于并排比较两个系统的评估得分
console.print(multi_agent_evaluation.model_dump())
```

> **输出结果讨论：** 评审模型的评分为我们之前的假设提供了定量证据。 **多智能体团队**的报告将获得显著更高的分数，尤其是在 清晰度与结构得分**和**分析深度得分 这两项上。评审模型的评语很可能会肯定报告中清晰的分段结构以及各部分内详尽、专家级的分析，这与通才型智能体产出**相对泛化且混杂**的输出形成了鲜明对比。 这一评估结果证实，对于**可以分解为多个专业领域的复杂任务**而言，多智能体架构在生成**高质量、结构化且可靠**的结果方面，是一种更优越的方案。

结论

在本节笔记本中，我们清晰地展示了**多智能体系统**相较于单一通才型智能体，在处理**复杂、多维度**任务时的显著优势。通过构建一个由**专业智能体**组成的团队——每个智能体都拥有**聚焦的角色定位和专长**，并辅以一名**管理者**来整合他们的成果——我们最终产出了质量明显更高的结果。

核心启示在于**专业化**的力量。正如在人类组织中一样，将大问题分解并交由**各自领域的专家**处理，能够带来更优的结果。尽管这种架构在协调方面引入了**更高的复杂性**，但其在最终输出的**结构性、深度和专业性**上的显著提升，使其成为任何需要在**多领域交付专家级表现**的严肃智能体应用中不可或缺的设计模式。